基于 Lean 4 的素数无界性形式化证明

*Agent 设计与实现报告 • 2026-10-25

一、任务

把命题「任意正整数都存在比它大的素数」形式化到 Lean 4 中，并围绕这一任务设计一个可复用的 agent，最终以 A4 报告呈现。

交付物分三部分：

- Agent 定义：`.opencode/agents/lean-prover.md`
- Lean 工程：`lean-euclid/`
- 本报告

二、数学命题与证明

2.1 命题

对任意正整数 n，存在素数 p，使得 $p > n$。

这是欧几里得「素数无穷多」的无界形式：它不仅断言素数无限，还给出一个可操作的构造性论证。

2.2 Euclid 反证法

设 $n \geq 1$，令

$$N = n! + 1$$

论证分三步。

第一步：$N > 1$。因为 $n! \geq 1$，所以 $N \geq 2$。每个大于 1 的自然数都有素因子，于是存在素数 p 使 $p \mid N$。

第二步：断言 $p > n$。反设 $p \leq n$。由于 p 是素数且 $p \leq n$，p 必整除 n!。而 p 又整除 $N = n! + 1$，因此 p 整除二者的差：

$$p \mid (n! + 1) - n! = 1$$

第三步：$p \mid 1$ 意味着 $p = 1$，但素数的定义要求 $p \geq 2$，矛盾。所以反设不成立，$p > n$。

结论：p 就是所求的、比 n 大的素数。

2.3 一个细节

$n = 1$ 时 $N = 2$，取 $p = 2 > 1$，结论同样成立。所以「正整数」这个限制只是为了贴合题面；定理实际上对一切自然数 n 都成立，在 Lean 里可以去掉正性假设。

三、Lean 实现

3.1 形式化语句

```lean
theorem exists_prime_gt_of_pos (n : ℕ) (_hn : 0 < n) :
∃ p : ℕ, p.Prime ∧ n < p
```

选用 ℕ 而不是 ℤ 或 ℝ：命题只涉及整除与大小比较，自然数是最弱、最自然的结构。

3.2 证明代码

```lean
import Mathlib

namespace EuclidWhale

theorem exists_prime_gt_of_pos (n : ℕ) (_hn : 0 < n) :
    ∃ p : ℕ, p.Prime ∧ n < p := by
  obtain ⟨p, hp_ge, hp_prime⟩ := Nat.exists_infinite_primes (n + 1)
  exact ⟨p, hp_prime, by omega⟩

theorem exists_prime_gt (n : ℕ) : ∃ p : ℕ, p.Prime ∧ n < p :=
  exists_prime_gt_of_pos n (Nat.zero_lt_succ n)

theorem infinite_setOf_prime : Set.Infinite {p : ℕ | p.Prime} :=
  Nat.infinite_setOf_prime

end EuclidWhale
```

3.3 说明

- `Nat.exists_infinite_primes` 是 Mathlib 中已形式化的 Euclid 定理：对任意 m，存在素数 p ≥ m。
- 取 m = n + 1，得到 p ≥ n + 1，再由 `omega` 推出 n < p。
- 三个定理分别是：带正性假设的题面版本、去掉假设的加强版、以及集合论形式。
- 全部依赖 Mathlib，没有自造公理，也没有 `sorry`。

四、Agent 设计

4.1 定位

`lean-prover` 是一个「形式化工程师」agent：输入是自然语言的数学命题，输出是可编译的 Lean 4 定理与一份如实的验证报告。

它既可以作为主 agent 直接使用，也可以作为子 agent 被其他 agent 调用来验证引理。

4.2 配置要点

- 位置：`.opencode/agents/lean-prover.md`
- 模式：`all`，primary 与 subagent 均可
- 权限：读写文件与联网查询默认允许；shell 默认需审批，但 `lake`、`lean`、`elan` 前缀的命令直接放行，避免每次编译都打断
- 模型：未指定，继承调用方会话的模型

4.3 工作流

1. 复述命题，列出全部假设、量词与边界情况。
2. 选择编码（ℕ / ℤ / ℝ / Finset / Set），并说明与原始表述的差异。
3. 选定导入，先用 `import Mathlib` 跑通，再用 `#min_imports` 收窄并重新验证。
4. 先搜索已有引理（`exact?`、`apply?`、`rw?`、`simp?`、loogle），再考虑自己证。
5. 写证明，给中间步骤起有意义的名字。
6. `lake build`，读错误、修、重来，直到零错误且无 `sorry` 警告。
7. `#print axioms` 审计，确认没有 `sorryAx`。
8. 生成 A4 报告。

4.4 关键约束

这是整个 agent 设计的核心。一个形式化 agent 最大的风险不是证不出来，而是把没验证的东西说成验证过了。所以系统提示里把以下几条写成硬规则：

- 禁止 `sorry`、`admit`，禁止新增 `axiom` 或用 `unsafe` 绕过。
- 不得悄悄弱化命题；需要加假设时必须先说明并取得同意。
- 只有编译器接受了，才允许使用「已证明」这个说法。
- 不得粘贴自己没有真正运行过的构建日志。
- 报告必须明确区分「已验证」与「未验证」。

4.5 失败处理

证不出来时，agent 必须报告失败点的精确目标状态、尝试过的路径及失败原因，并区分「命题为假」与「我没找到证明」。仓库应尽量保持在可编译状态；若留下 `sorry`，源码与报告都要标注为未完成。

五、验证状态

本次按需求选择「只写代码，不安装不编译」，因此必须如实声明：

- 未安装 Lean 工具链，未执行 `lake build`。
- 上述 Lean 代码未经编译器验证。
- 证明思路与第二部分的数学论证一致，引用的 `Nat.exists_infinite_primes` 是 Mathlib 中的既有定理；但不排除语法细节或引理名与当前 Mathlib 版本不匹配的可能。

这恰好是 `lean-prover` 要防的那种情况，所以这里不给「已证明」的结论，只给「待验证」。

5.1 复现步骤

安装 Lean 与 Mathlib 后执行：

```bash
lake exe cache get
lake build
```

若构建通过，再检查文件末尾 `#print axioms` 的输出：应当只包含 `propext`、`Classical.choice`、`Quot.sound` 等标准公理，不出现 `sorryAx`。

六、局限与后续

- 未验证是当前最大的局限，安装工具链后应第一时间补做。
- `lakefile.lean` 中 Mathlib 固定到 `master`，而 `lean-toolchain` 写的是 `stable`。实际使用时必须把 `lean-toolchain` 换成 Mathlib 仓库根目录中的版本，否则 Lake 会报版本不匹配。
- 当前只证明了存在性，没有给出「第 n 个素数」之类的构造算法；若要可计算版本，需要改用 `Nat.nth` 或给出显式搜索过程。
- Agent 的权限设计偏宽松，文件读写全部允许。若用于不受信任的输入，应收窄 `edit` 的 `resource` 范围。

七、结论

数学上，命题成立，证明是经典的 Euclid 构造：取 n! + 1 的素因子，它必然大于 n。

工程上，`lean-prover` 把「形式化」拆成可执行的八步流程，并用五条硬规则约束模型不要虚报验证状态。本次交付的 Lean 代码逻辑完整但未编译，报告据此标注为待验证——这本身就是该 agent 设计要保证的行为。